CODE for Causal Inference in Biomedical Imaging via Functional Linear Structural Equation Models

This is the ReadMe file for the estimation and testing procedures presented in the paper.

Summary of the paper

Understanding the causal effects of organ-specific features from medical imaging on clinical outcomes is essential for biomedical research and patient care. We propose a novel Functional Linear Structural Equation Model (FLSEM) to capture the relationships among clinical outcomes, functional imaging exposures, and scalar covariates like genetics, sex, and age.

$$Y_i = \sum_{\ell=1}^{p} X_{i\ell}\beta_\ell + \int_{\mathcal{T}} \mathbf{Z}_i(t)\mathbf{B}(t)dt + \epsilon_i \quad \text{and} \quad \mathbf{Z}_i(t) = \sum_{\ell=1}^{p} X_{i\ell}C_\ell(t) + \mathbf{E}_i(t).$$

Traditional methods struggle with the infinite-dimensional nature of exposures and complex covariates. Our FLSEM overcomes these challenges by establishing identifiable conditions using scalar instrumental variables. We develop the Functional Group Support Detection and Root Finding (FGSDAR) algorithm for efficient variable selection, supported by rigorous theoretical guarantees, including selection consistency and accurate parameter estimation. We further propose a test statistic to test the nullity of the functional coefficient $\mathbf{B}(t)$, establishing its null limit distribution.

The empirical application in the paper uses UK Biobank imaging, genetics, and cognitive data. Individual-level UK Biobank data are restricted and are not redistributed with this code. The files in this folder therefore provide the FLSEM implementation and a synthetic demonstration workflow that reviewers and readers can run without UK Biobank access.

Reproducibility scope

The materials in this folder reproduce the computational method on simulated or demonstration data. They implement the estimation and testing procedures used in the paper, including the first-stage functional exposure model, the outcome model, and the null-effect test for the functional coefficient.

The supplied demo does not reproduce the full UK Biobank empirical tables and figures by itself, because those analyses require approved access to restricted UK Biobank data and the secure preprocessing workflow for phenotypes, SNPs, genetic principal components, and white-matter tract profiles. The manuscript and supplement document the empirical variable mapping: seven cognitive outcomes, twenty-one white-matter tracts summarized on a one-hundred-point grid, and screened SNP/covariate sets.

The full Monte Carlo tables in the manuscript can be regenerated by wrapping the supplied functions in the simulation loops described in the main text and supplement. The short demo below is intended as an executable check of the code interface and core FLSEM calculations.

Contents of this folder

- Gen_data_new.R: generate simulated data. Detailed descriptions of all model parameters and their roles are provided within the file. The returned list contains `y`, `x`, `z`, `x_aux`, `beta_true`, `C_all`, `gamma`, `K_gram`, `idx_true_X`, and `idx_true_Y`. In the demo, `y` is the scalar outcome, `x` is the functional exposure observed on a grid, and `z` is the matrix of scalar covariates.

- LSEM_estimation.R: implements the main estimation procedure. It orchestrates the overall workflow by calling the following components. This function performs the first-stage functional exposure model, constructs fitted functional exposures, fits the outcome model, and returns selected variables, functional coefficient estimates, and simulation MSE summaries when truth is supplied.
  - functional_GroupL0_gradient.R: implements the estimation of the functional exposure model using a group-wise L_0-type regularization approach. This is the FGSDAR-style first-stage routine for selecting scalar variables associated with the functional exposure.
  - divide_conquer_procedure.R: implement a divide-and-conquer strategy to improve the computational efficiency of the functional exposure model estimation. This optional routine is intended for large two-dimensional or high-resolution functional domains and uses multicore parallelization through `foreach` and `doParallel`.
  - PFLR_RKHS_estimate.R: perform estimation of the outcome model using a penalized functional linear regression framework. It estimates the functional coefficient `B(t)`, scalar coefficients, and selected scalar covariates after the fitted functional exposure is formed.
- pvalue_cal.R: conduct testing for the nullity of the functional exposure coefficient. It returns the test statistic, the chi-square approximation parameters `kappa` and `zeta`, the standardized statistic, and the p-value.
- demo_data.Rdata: Synthetic demonstration data saved as an R object named `data`. It has `n = 200` observations, `x` with dimension 200 by 100, `z` with dimension 200 by 20, `beta_true` of length 100, `C_all` with dimension 5 by 100, and `gamma` of length 20.
- ReadMe.Rmd and ReadMe.pdf: Documentation and runnable example code for the reproducibility materials.

Software requirements

Primary software: R. The manuscript reports R 3.6.0 for the simulation studies; the demo code is written to run in R 3.6.0 or later, subject to package availability.

Required R packages for the demo are `MASS`, `Matrix`, and `refund`. The optional divide-and-conquer routine additionally uses `foreach` and `doParallel`. On the environment used to inspect these materials, the installed versions were `MASS` 7.3-56, `Matrix` 1.5-3, `refund` 0.1-26, `foreach` 1.5.2, and `doParallel` 1.0.17.

Run the code from this Code directory so that the relative `source()` calls and `demo_data.Rdata` path resolve correctly.

Data used by the demo

The demo can either load `demo_data.Rdata` or regenerate a synthetic data set with `Gen_data_new()`. The synthetic data are generated from the simulation model in the paper and are not derived from UK Biobank participants.

Demo code for estimation and testing

Preparations

```
source('Gen_data_new.R')
source('LSEM_estimation.R')
source('PFLR_RKHS_estimate.R')
source('functional_GroupL0_gradient.R')
source('divide_conquer_procedure.R')
source('pvalue_cal.R')

library(refund)
```

The following code sets a random seed for reproducibility. Users may either regenerate synthetic data or load the supplied `demo_data.Rdata` object.

Generate or load demo data

```
set.seed(20240417)

n <- 200
pn <- 20

kp <- 1 ## for Gaussian kernel

lent <- 100
locations <- seq(0, 1, length = lent)

lambda_range <- seq(1e-2, 1e-3, length = 5)

sp_range <- c(5:10)

init_gamma <- rep(0, pn)
max_iter <- 50
n_basis <- n

#### The kernel ######
kp <- 1

K_gram <- outer(locations, locations, function(x, y) exp(- (x-y)^2 / kp))

B <- 1

rho2 <- 0.3
cor <- 0.2

data <- Gen_data_new(n, pn, locations, rho2, cor, B)

## Alternative:
## load("demo_data.Rdata")
```

The generated object `data` contains the scalar outcome `data$y`, the functional exposure matrix `data$x`, the scalar covariate matrix `data$z`, true simulation coefficients, and support indicators used only for checking demo MSEs. In real data applications, `beta_true` and `gamma` are unknown and can be set to `NA` or omitted where MSE summaries are not needed.

Estimation and testing

```
######## Estimation. beta_true and gamma true are used to calculate MSEs, can be set to NA in real data
t_start <- Sys.time()
est <-  LSEM_estimation(data$y, data$x, data$z, K_gram, data$beta_true, data$gamma)

# Functional coefficient estimation
est$final_results$beta_est
```

```
##   [1] 0.8278924 0.8287783 0.8294645 0.8299498 0.8302330 0.8303132 0.8301897
##   [8] 0.8298616 0.8293285 0.8285900 0.8276456 0.8264954 0.8251392 0.8235773
##  [15] 0.8218098 0.8198371 0.8176599 0.8152787 0.8126944 0.8099079 0.8069203
##  [22] 0.8037329 0.8003469 0.7967639 0.7929854 0.7890133 0.7848495 0.7804959
##  [29] 0.7759546 0.7712281 0.7663186 0.7612286 0.7559609 0.7505181 0.7449031
```

```
##  [36] 0.7391190 0.7331687 0.7270555 0.7207828 0.7143538 0.7077722 0.7010414
##  [43] 0.6941653 0.6871475 0.6799920 0.6727028 0.6652837 0.6577389 0.6500727
##  [50] 0.6422892 0.6343927 0.6263876 0.6182783 0.6100693 0.6017651 0.5933703
##  [57] 0.5848893 0.5763270 0.5676879 0.5589767 0.5501981 0.5413569 0.5324577
##  [64] 0.5235054 0.5145047 0.5054603 0.4963770 0.4872596 0.4781127 0.4689411
##  [71] 0.4597494 0.4505424 0.4413247 0.4321009 0.4228756 0.4136533 0.4044385
##  [78] 0.3952356 0.3860492 0.3768834 0.3677427 0.3586311 0.3495530 0.3405124
##  [85] 0.3315133 0.3225597 0.3136555 0.3048044 0.2960103 0.2872767 0.2786072
##  [92] 0.2700054 0.2614745 0.2530178 0.2446387 0.2363401 0.2281251 0.2199965
##  [99] 0.2119573 0.2040101
```

```
# Functional coefficient MSE
est$MSE_beta
```

```
## [1] 0.005664676
```

```
# Selected scalar variables in the exposure and outcome models
which(est$ind_for_X)
```

```
## [1] 1 2 3 4 5
```

```
which(est$ind_for_Y)
```

```
##  [1]  1  6  7  8  9 10 13 14 17 18
```

```
####### Testing
functional_fitted <- data$z %*% est$C_fun_all
Z_for_Y <- data$z[, est$ind_for_Y]

test_result <- pvalue_cal(data$y, functional_fitted, Z_for_Y, K_gram)

# p-value for testing the nullity of the functional coefficient
test_result["pvalue"]
```

```
## pvalue
##      0
```

```
print(Sys.time() - t_start)
```

```
## Time difference of 9.799403 secs
```

Expected output

The demo prints an estimated functional coefficient vector, a simulation MSE for the functional coefficient, the selected scalar variables in the exposure and outcome models, and a p-value for the null functional effect test. Because the demo uses random simulation and iterative variable selection, exact numeric values may vary across R versions, BLAS/LAPACK libraries, and random seeds if the seed is changed.

Notes for adapting the code

For a real data analysis, replace `data$y`, `data$x`, and `data$z` with the processed outcome vector, functional exposure matrix, and scalar covariate matrix. The matrix `K_gram` should be chosen to match the functional domain and reproducing kernel used in the analysis. For the UK Biobank white-matter application, the functional exposure corresponds to log-quantile-density transformed FA tract profiles on a common grid.

The function `LSEM_estimation()` currently relies on several tuning objects defined in the calling environment, including `lambda_range`, `sp_range`, `max_iter`, `init_gamma`, `locations`, and `pn`. These are defined explicitly in the demo above and should be set before calling the function in new applications.

The optional divide-and-conquer path `divide_x = TRUE` is intended for larger functional domains. It additionally requires `locations_1`, `locations_2`, `locations`, and a suitable number of cores through the `num_core` argument inside `functional_divide_conquer()`.